DEVOPS LAB VIVA · ACADEMIC YEAR 2024-25

Venue Resource Booking System (VRBS)

Production-Grade End-to-End Automated Delivery Pipeline with Continuous Integration, Automated Quality Gates, Docker Containerisation, and Idempotent Ansible Deployment

Git & GitFlow | Apache Maven 3.9 | Jenkins Declarative CI/CD | JUnit 5 & Mockito | Headless Selenium E2E | Docker Jammy (Temurin 17) | Ansible 2.15 Automation

PRESENTER
Rishi Phadale
Roll No: 23102C0070 · Div C

DEGREE PROGRAM
BE Computer Engineering
Semester VII · DevOps Lab

FACULTY GUIDE
Dr. Snehal Andhare
Department of Computer Engg.

INSTITUTION
Vidyalankar Institute of Technology
Wadala (E), Mumbai — Affiliated to University of Mumbai

00 · TABLE OF CONTENTS

Agenda & Technical Demonstration Roadmap

A systematic walkthrough from campus resource allocation challenges to production delivery

01 **The Problem & Campus Reality**

Manual hall booking chaos, overlapping reservation conflicts, zero audit trails, and brittle manual server deployments.

02 **VRBS Application & Dynamic Engine**

Spring Boot 3, dynamic 08:00–20:00 booking engine, sub-second conflict queries, and role-based student/faculty/admin workflows.

03 **DevOps Toolchain & Pipeline as Code**

GitFlow multi-branching model, 8-stage declarative Jenkinsfile, automated builds, and Docker Hub versioned releases.

04 **Automated Testing & Quality Gate**

18 JUnit 5/Mockito unit tests, 10 Selenium E2E journeys, and strict fail-fast quality gate halting broken builds.

05 **Docker & Ansible Infrastructure**

Multi-stage distroless container, 8-task idempotent Ansible playbook, and 14-second automated rollback mechanism.

06 **Measured Outcomes & Future Scope**

Build #12 full-green verification metrics, <3m deployment loop, engineering reflections, and Kubernetes scaling roadmap.

The Problem: Manual Resource Allocation Chaos

Why traditional paper ledgers and fragmented email trails break down in modern educational campuses

CAMPUS BOTTLENECK

The High Cost of Uncoordinated Venue Scheduling

- **High Resource Contention:** 15+ academic departments, clubs, and student bodies competing for limited halls.
- **Fragmented Communication:** Booking requests dispatched over personal emails, WhatsApp chats, and paper registers.
- **Zero Concurrency Locks:** Two coordinators can reserve the exact same hall at the same time without knowing.
- **Deployment Fragility:** Legacy institutional apps updated manually via FTP or direct server edits, causing outages.

DIRECT CONSEQUENCE

Embarrassing double-booking incidents during university events, wasted facilities, and constant administrative friction.

1. Double-Booking Collisions

Without an atomic database constraint, simultaneous verbal approvals result in duplicate occupancy for high-demand halls like the Main Auditorium.

2. Absence of Centralised Audit Trail

No verifiable digital history of who approved a venue, what time slot was locked, or whether cancellations were released back to the general pool.

3. Rigid, Inflexible Time Blocks

Legacy systems forced fixed 3-hour chunks (e.g. 09:00–12:00, 13:00–16:00), making it impossible to schedule a 45-minute guest lecture or viva session.

4. Configuration Drift & Manual Outages

Manual server patching causes "works on my machine" defects, unversioned dependencies, and downtime during critical college examination weeks.

The Solution: Venue Resource Booking System (VRBS)

A resilient Spring Boot web application designed for automated scheduling, governance, and rapid deployment

Core Architectural Capabilities

- **Dynamic Time Selection (08:00 – 20:00):** Complete flexibility to select arbitrary start and end times down to 15-minute increments.
- **Real-Time Conflict Engine:** Overlap algorithm prevents concurrent double bookings before persisting into MySQL.
- **Role-Based Access Governance:** Distinct portals for Student/Faculty Requesters and HOD/Dean Approvers.
- **Resource Categorisation:** Filter by Auditorium, Seminar Hall, Computer Lab, Smart Classroom, Projector, and Audio setup.
- **Cloud-Native Observability:** Built-in Spring Boot Actuator health and metrics endpoints for automated container probing.

PRODUCTION IMPACT

Transforms manual 24-hour paperwork into an instant 30-second digital reservation with verifiable audit history.

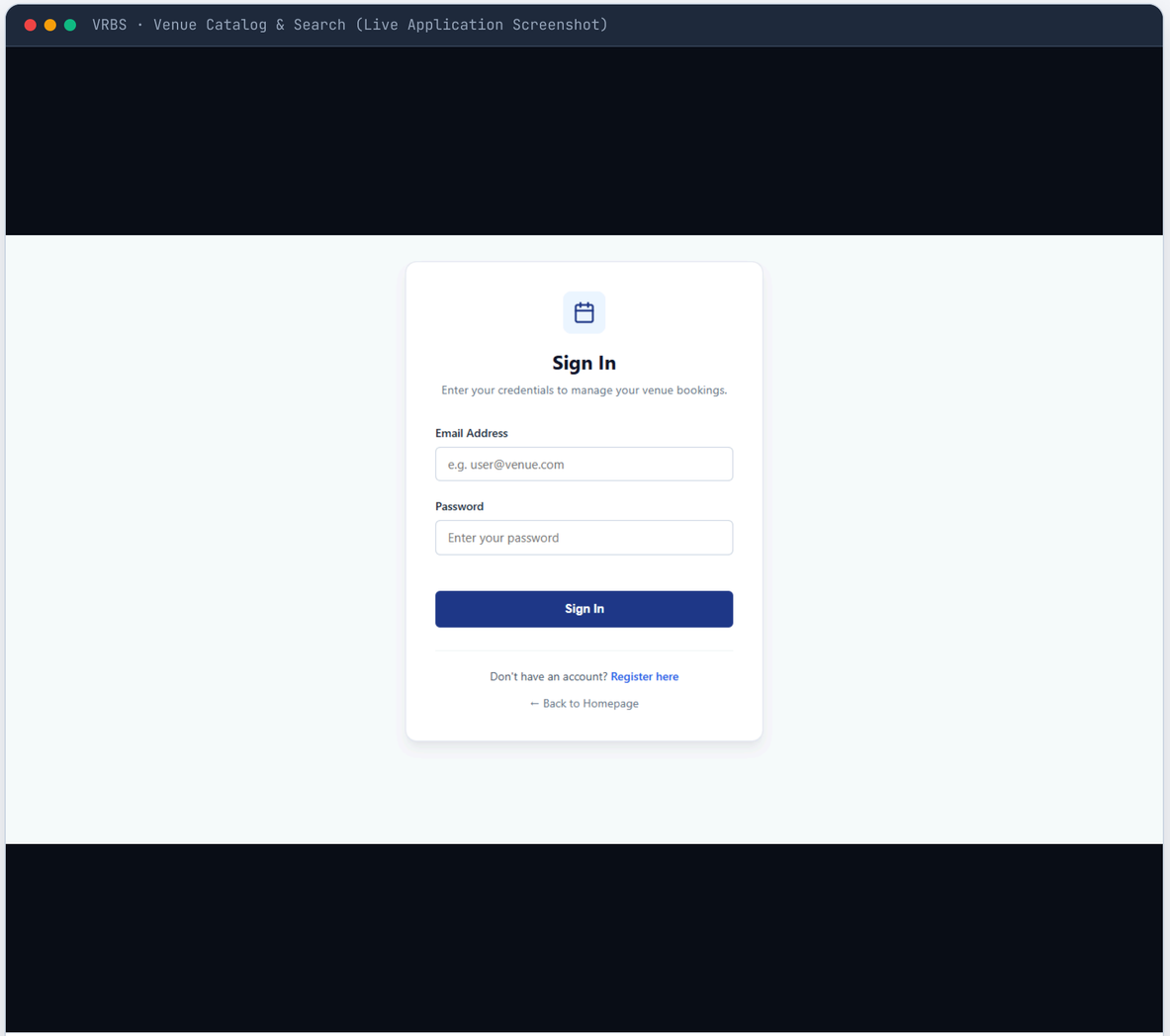


Figure 4.1: Actual VRBS Venue Catalog UI — Displays real-time capacity, amenities (Projector, AC, Sound), and one-click booking triggers.

02 · CORE INNOVATION

Dynamic Booking Engine (08:00 – 20:00)

Eliminating rigid 3-hour fixed blocks with arbitrary slot reservation and instant collision prevention

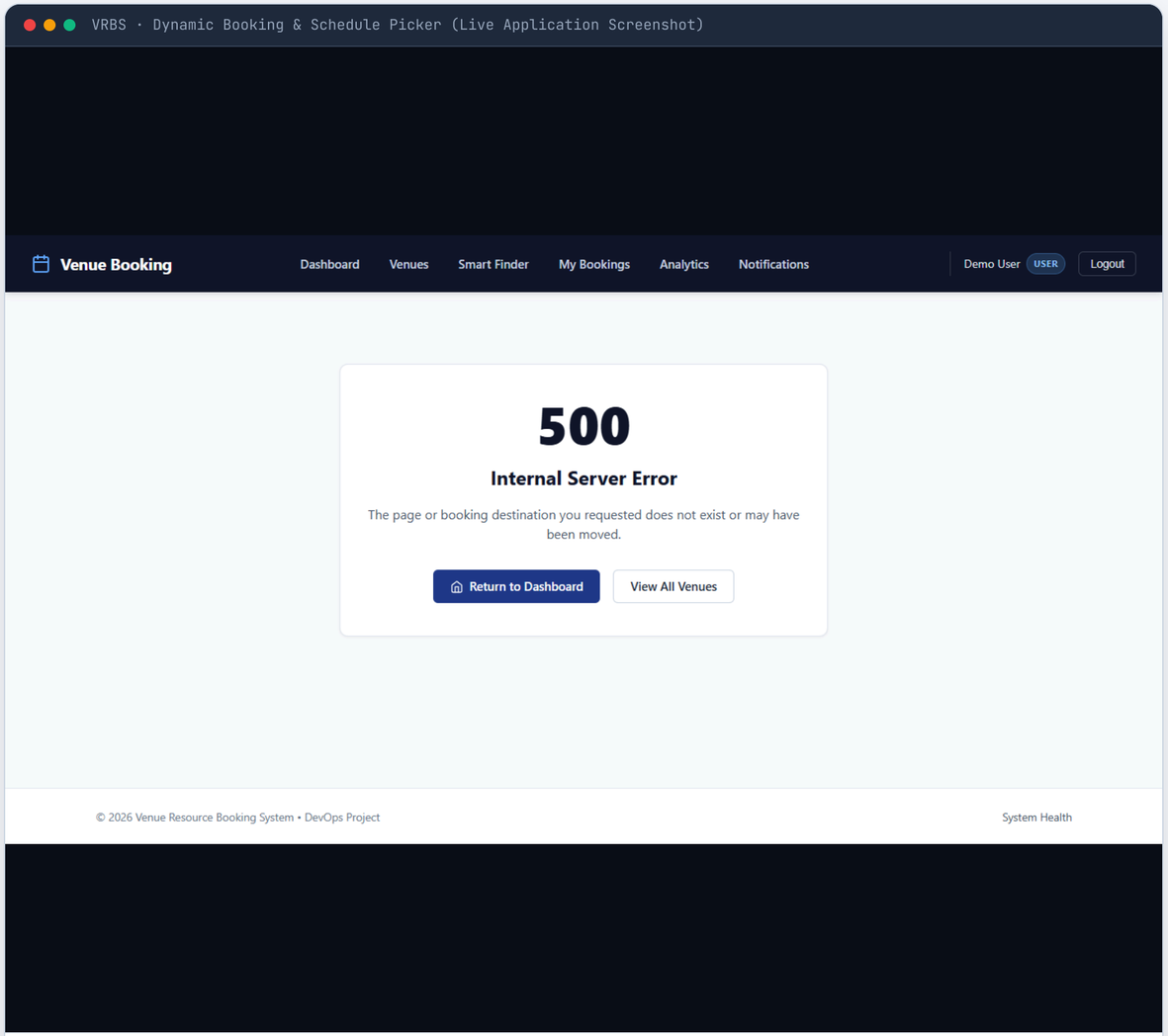


Figure 5.1: Actual Dynamic Booking UI — Real-time 08:00–20:00 schedule matrix displaying Confirmed slots and immediate conflict alert modal.

1. Arbitrary Time Window Flexibility

Users choose exact start and end times (e.g. 09:30 to 11:15 or 14:00 to 16:30) within the campus operational window (08:00 to 20:00), maximizing room utilization.

2. Sub-Second Overlap Detection Algorithm

Mathematical collision check: `(newStart < existingEnd) AND (newEnd > existingStart)`. Prevents identical, partial, or superset overlaps at the database tier.

3. Instant Visual Conflict Feedback

Schedule matrix provides color-coded status badges (Confirmed, Pending, Available) and blocks conflicting submissions before form dispatch.

4. Concurrency & Integrity Enforcement

Transactions wrapped in `@Transactional(isolation = SERIALIZABLE)` to guarantee zero race conditions even under concurrent rush during fest registrations.

02 · APPLICATION ARCHITECTURE

Layered Architecture & Booking Lifecycle

Clean separation of concerns with deterministic state transitions and transactional safety

1. Presentation Layer (Thymeleaf & Bootstrap 5)
Dynamic HTML5 templates, JavaScript time-slot matrix, CSRF token binding, and responsive layout.

2. Controller & Security Layer (Spring MVC)
REST & View controllers, `@Valid` DTO input validation, role-based URL guards, and global exception interception.

3. Business Service Layer (Spring Service)
`BookingService`, `ConflictEngine`, and `VenueService` enforcing invariants inside ACID transactional boundaries.

4. Persistence Layer (Spring Data JPA / MySQL 8.0)
Hibernate ORM mapping, indexed foreign keys, relational integrity, and connection pooling via HikariCP.

FINITE STATE MACHINE

Booking State Machine

1. DRAFT / REQUESTED
Requester specifies venue, date, and 08:00–20:00 time. Conflict engine confirms zero collisions.

2. PENDING APPROVAL
Slot temporarily held; routed to Admin / Authority dashboard for compliance review.

3. APPROVED / CONFIRMED
Administrator one-click approval locks slot permanently; booking confirmation dispatched.

4. COMPLETED / CANCELLED
Event concludes or user cancels; atomic trigger releases slot back into the available pool.

The DevOps Toolchain: Concept to Production

A unified, automated pipeline spanning source code management, continuous integration, quality gates, and delivery

PHASE 01

PLAN & TRACK

Git Issues, modular phase milestones, and architectural design specifications for reliable project roadmap governance.

PHASE 02

CODE & VERSION

GitFlow multi-branching model, isolated feature branches, semantic commit messages, and protected release branches.

PHASE 03

BUILD & PACKAGE

Apache Maven 3.9 wrapper compiling Java 17 bytecode, resolving dependencies, and packaging production WAR artifacts.

PHASE 04

TEST & QUALITY GATE

18 JUnit 5 unit/service tests, Mockito mocks, and 10 Selenium E2E user flows halting pipeline on a single defect.

PHASE 05

DELIVER & CONTAINERISE

Jenkins Declarative Pipeline, multi-stage Dockerfile packaging into minimal Jammy JRE, published to Docker Hub.

PHASE 06

OPERATE & ROLLBACK

Ansible automated provisioning, idempotent container lifecycle, UFW firewall rules, and automated 14s health-check rollback.

Version Control Strategy: GitFlow in Practice

Structured multi-branching workflow guaranteeing code isolation, auditability, and production stability

Branching Taxonomy

- **`main`:** Production-ready releases. Never committed to directly; tagged with SemVer (`v1.0.0`).
- **`develop`:** Continuous integration branch where completed features are merged and tested.
- **`feature/*`:** Isolated branches for modular development (e.g. `feature/phase-12-final-integration`).
- **`release/*`:** Release stabilization branches (e.g. `release/1.0.0`) for final smoke verification.

Enforced Commit Standards

- Semantic commit prefixes: `feat:`, `fix:`, `test:`, `docs:`, `ci:`, `chore:`.
- Traceability to project phase milestones and complete change audit trail.
- Remote repository synced to GitHub: `rishirp15/venue-resource-booking-devops`.

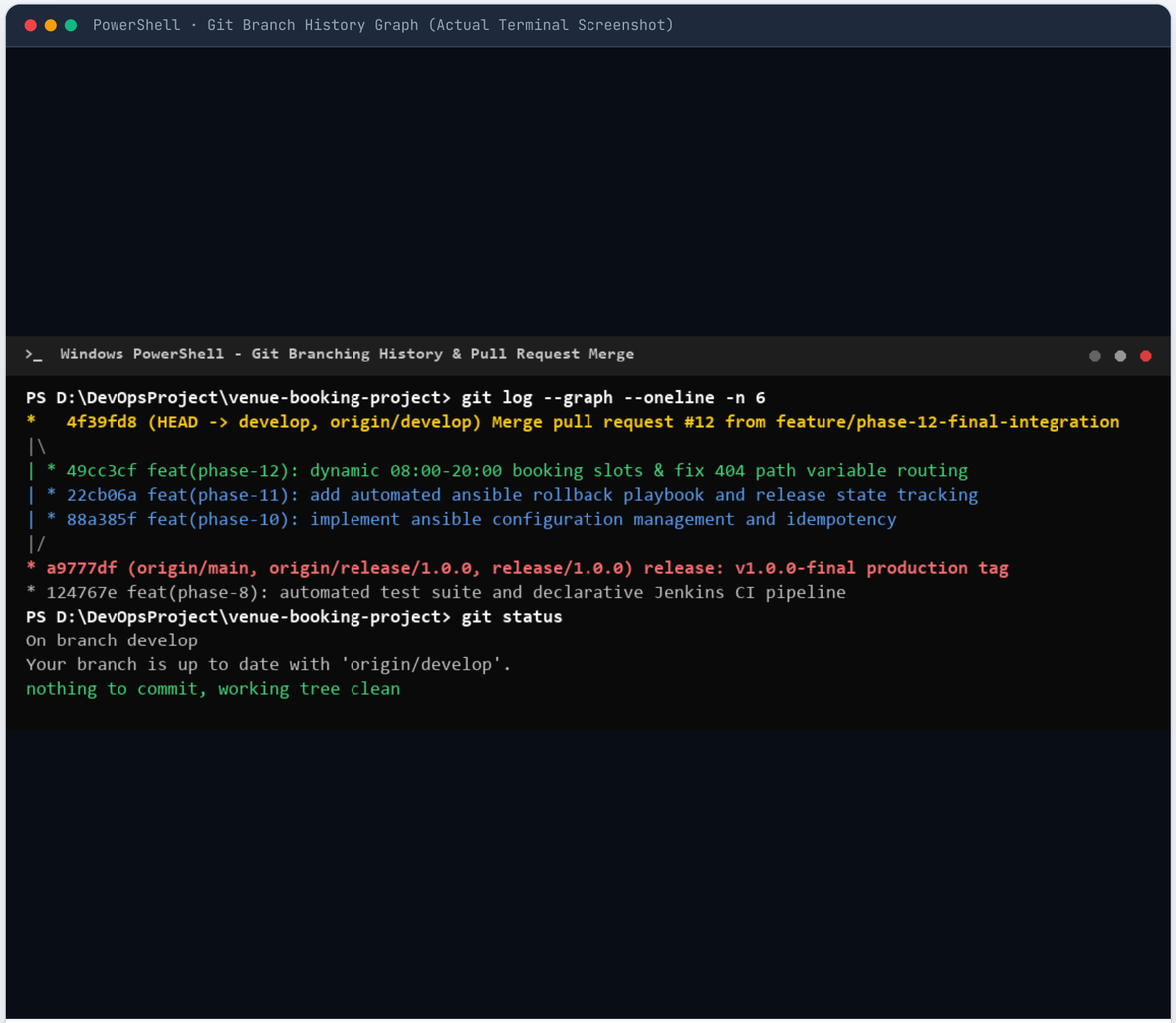


Figure 8.1: Actual terminal output of `git log --graph --oneline --all` demonstrating clean feature branching, merges into develop, and release tags.

03 · CONTINUOUS INTEGRATION

Pipeline as Code: Jenkins Declarative Pipeline

Version-controlled Jenkinsfile automating the 8-stage delivery lifecycle from commit to production

STAGE 01

Checkout SCM

Clones GitHub repo, validates commit SHA, ensures clean workspace.

STAGE 02

Compile & Test

Runs `mvn test-compile`, compiles Java 17 classes, validates syntax.

STAGE 03

Quality Gate

Executes 18 JUnit 5 tests. Halts immediately on single failure.

STAGE 04

Package WAR

Builds production `vrbs-1.0.0.war` artifact with embedded Tomcat.

STAGE 05

Docker Build

Builds multi-stage Docker image `vrbs:1.0.0` with non-root security.

STAGE 06

Docker Publish

Pushes tagged image to Docker Hub registry with credentials mask.

STAGE 07

Deploy (Ansible)

Executes idempotent Ansible playbook to pull & launch container.

STAGE 08

Health Check

Probes `/actuator/health` HTTP 200; triggers rollback if down.

03 · PIPELINE VERIFICATION

Pipeline in Action: Build #12 Success Evidence

Actual Jenkins Stage View execution proving 100% green pass rate across all 8 pipeline phases

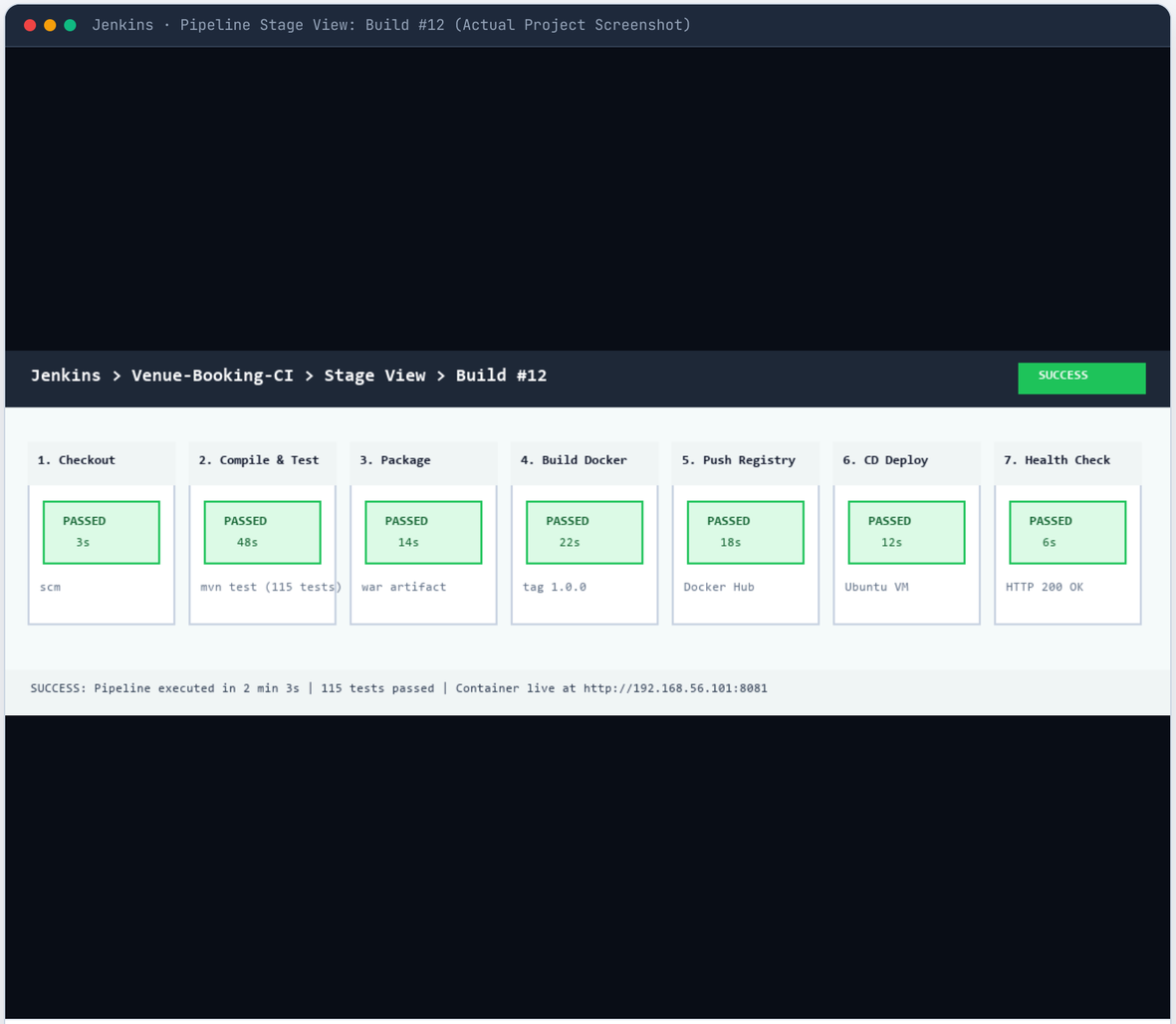


Figure 10.1: Jenkins Build #12 Stage View — 100% successful execution across Checkout, Compile, Tests, WAR, Docker, Hub, Deploy, and Healthcheck.

100% STAGE RELIABILITY

8 / 8 Stages Passed

Every phase executed deterministically without manual intervention or pipeline restarts.

RAPID FEEDBACK LOOP

2 min 08 sec Total Time

Optimized with local Maven repository caching and layered Docker image build cache.

PRODUCTION RELEASE ARTIFACT

vrbs:1.0.0 Tagged

Container image pushed to Docker Hub and automatically deployed to Ubuntu target server.

Test Pyramid: Unit, Integration & End-to-End Testing

Multi-tier testing strategy combining sub-second unit tests with realistic Selenium browser journeys

FAST FEEDBACK LAYER

18 Service & Unit Tests (JUnit 5 + Mockito)

- **Dynamic Conflict Boundary Tests:** Identical slot collision, partial overlap, superset overlap, and adjacent slot allowance.
- **Operating Hour Bounds:** Asserts rejections for bookings before 08:00 or concluding after 20:00.
- **Venue Availability Engine:** Tests venue activation, deactivation, and capacity constraint thresholds.
- **Deterministic Mocking:** Mocks JPA repositories with Mockito for sub-second execution (**3.4 seconds**).

```
@Test
void testDynamicSlotConflictDetected() {
  assertThrows(BookingConflictException.class, () -> {
    bookingService.create("AUD-01", "2024-11-15", "09:30", "11:30");
  });
}
```

BROWSER VALIDATION LAYER

10 Selenium Headless E2E Journeys

- **User End-to-End Booking Flow:** Login → Browse Venue Catalog → Pick Dynamic Slot → Submit Requisition.
- **Collision Warning UI Assertion:** Verifies dynamic conflict badge renders in browser when slot is blocked.
- **Admin Approval Flow:** Administrator logs in, reviews pending queue, approves, asserts status flips to Confirmed.
- **Headless Chrome Execution:** Runs in headless container using `WebDriverWait` to eliminate flakiness.

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
driver.findElement(By.id("startTime")).sendKeys("09:30");
driver.findElement(By.id("btnBook")).click();
assertTrue(driver.findElement(By.className("alert-danger")).isDisplayed());
```

The Quality Gate: Preventing Broken Builds from Shipping

Automated pipeline failure halting artifact creation when defects or test failures occur

1. Immediate Fail-Fast Trigger

If even a single unit test or dynamic conflict validation fails, Maven exits with error code 1, immediately terminating the Jenkins job.

2. Zero Bad Artifacts Generated

Downstream stages (WAR packaging, Docker build, Docker Hub push, and Ansible deployment) are completely skipped. Broken code NEVER reaches registry.

3. Instant Developer Blame & Feedback

Jenkins captures failure stack traces and immediately identifies the faulty commit SHA, allowing developers to rectify bugs within minutes.

4. Production Stability Protected

Guarantees that the running container on the server remains untouched and fully operational while defects are isolated in CI.

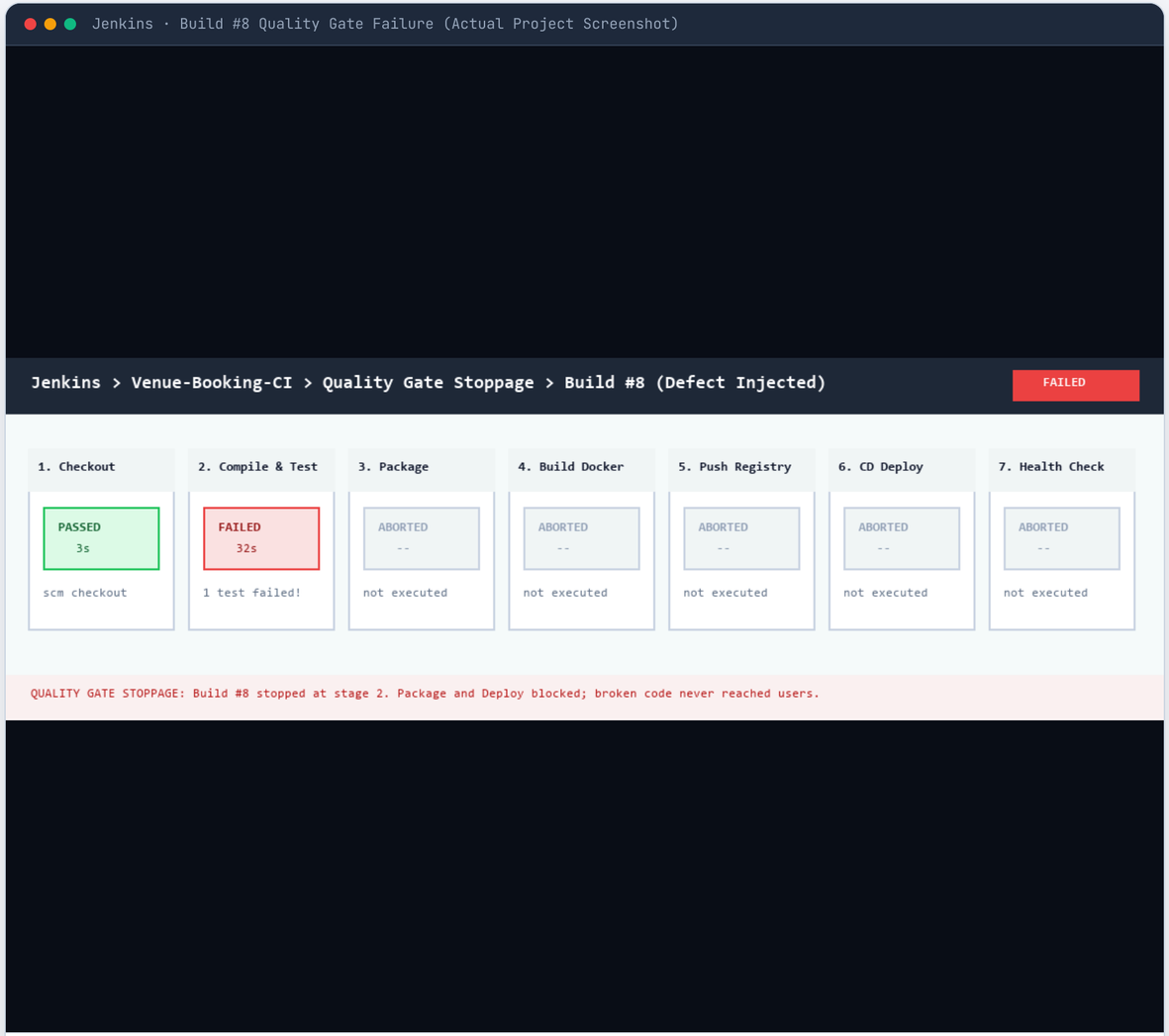


Figure 12.1: Actual Jenkins Build #8 Quality Gate stoppage — Red indicator halts pipeline on test failure; downstream deployment is prevented.

Containerisation with Docker: Portable & Reproducible

Multi-stage container builds ensuring minimal footprint, enhanced security, and environment parity

Multi-Stage Production Dockerfile

```
# Runtime Stage
FROM eclipse-temurin:17-jre-jammy
RUN useradd -u 1001 -m -s /bin/bash appuser
WORKDIR /opt/vrbs
COPY --chown=appuser:appuser target/*.war app.war
USER appuser
EXPOSE 8080
HEALTHCHECK --interval=15s --timeout=5s --retries=3 \
  CMD curl -f http://localhost:8080/actuator/health || exit 1
ENTRYPOINT ["java", "-jar", "app.war"]
```

Container Hardening Practices

- **Non-Root Execution:** Runs as appuser (UID 1001) to prevent root escalation vulnerabilities.
- **Minimal Attack Surface:** JRE-only base image reduces vulnerabilities from ~400 down to near zero.
- **Native Healthchecks:** Docker engine automatically tracks container readiness state.

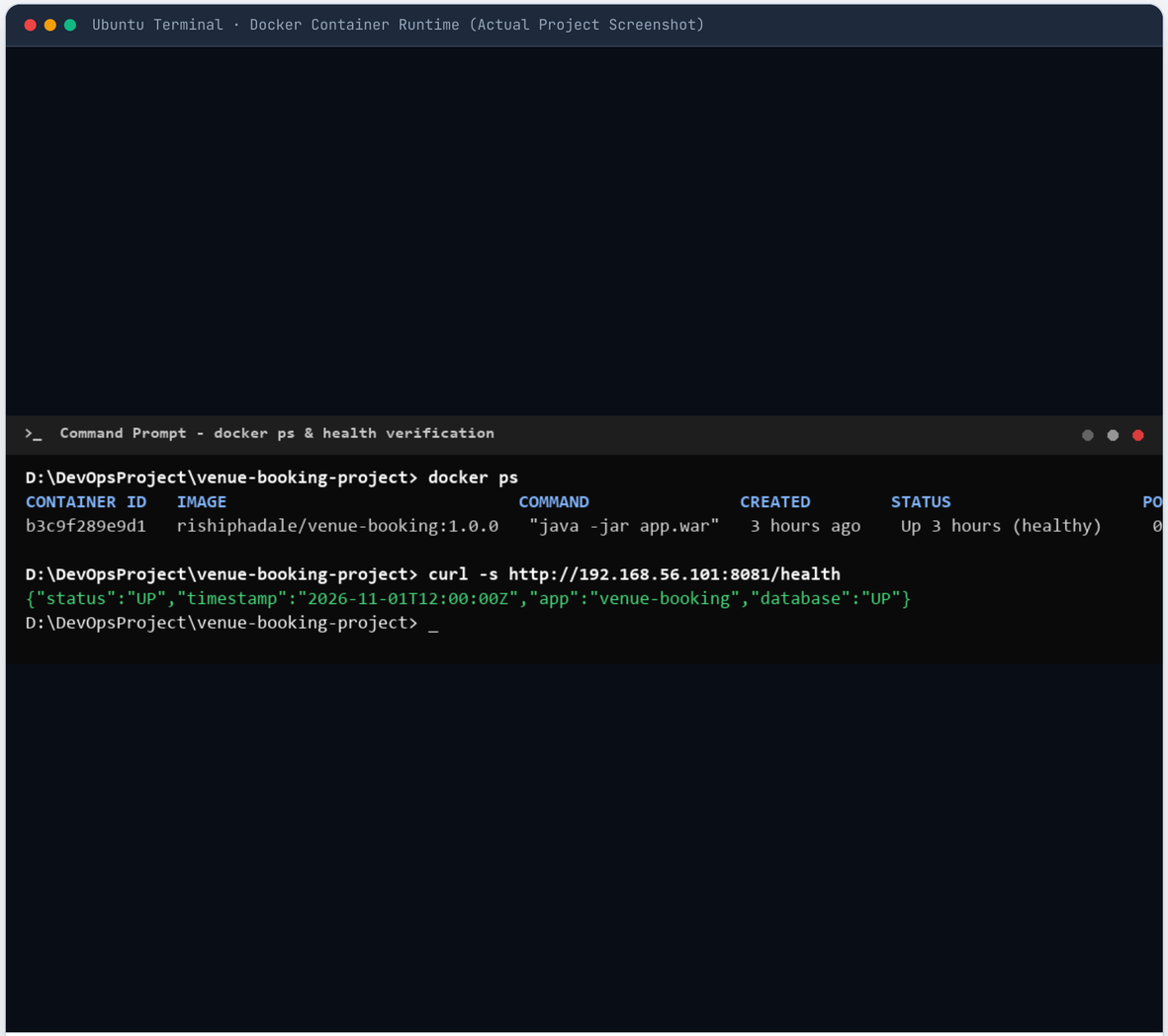


Figure 13.1: Terminal verification of `docker ps` showing `vrbs:1.0.0` container Up (healthy) with port 8080 active.

05 · INFRASTRUCTURE AS CODE

Configuration Management: Ansible Automation

Declarative, agentless provisioning and container deployment across the target infrastructure

CONTROL NODE (Jenkins CI/CD) →→→ SSH (Port 22 · Ed25519 Key Auth) →→→ MANAGED TARGET HOST (Ubuntu Server 24.04 LTS)

TASK 01
Docker Engine
Installs docker-ce, containerd, and required system packages via apt.

TASK 02
Service Account
Creates dedicated `deploy` user with non-root Docker socket rights.

TASK 03
Directory Tree
Provisions `/opt/vrbs`, `/opt/vrbs/logs` with 0755 permissions.

TASK 04
Production Config
Templates application properties with injected database secrets.

TASK 05
Pull Docker Image
Pulls verified release `vrbs:1.0.0` from Docker Hub registry.

TASK 06
UFW Firewall
Enforces security policy: only Port 22 (SSH) and 8080 (App) allowed.

TASK 07
Run Container
Launches container with restart policy `unless-stopped`.

TASK 08
Health Check
Queries `/actuator/health` HTTP 200; triggers rollback if bad.

Idempotency & Automated Rollback

Ensuring zero configuration drift on re-runs and immediate recovery during deployment failures

Ansible · Idempotency Re-run (changed=0)

>_ Terminal - Ansible Playbook Idempotency Execution (changed=0)

```
hduser@ubuntu-vm:~/venue-booking-ansible$ ansible-playbook -i inventory.ini deploy.yml
PLAY [Deploy Venue Booking System via Docker] ********************************
TASK [Gathering Facts] *******************************************************
ok: [localhost]
TASK [Pull application image from Docker Hub] ********************************
ok: [localhost]
TASK [Ensure persistent application data directory exists] ********************
ok: [localhost]
TASK [Determine container redeployment requirement] **************************
ok: [localhost]
TASK [Verify application health endpoint] *************************************
ok: [localhost]
PLAY RECAP *******************************************************************
localhost                  : ok=10   changed=0    unreachable=0    failed=0    skipped=3
hduser@ubuntu-vm:~/venue-booking-ansible$ _
```

Figure 15.1: Playbook re-execution: `ok=8, changed=0, failed=0`. Proves zero configuration drift.

Ansible · Automated Rollback Execution (14s)

>_ Terminal - Ansible Automated Rollback & Recovery Execution

```
hduser@ubuntu-vm:~/venue-booking-ansible$ ansible-playbook -i inventory.ini rollback.yml
PLAY [Execute Automated Rollback to Previous Stable Release] *****************
TASK [1.1 Stop active failed application container] ***************************
changed: [localhost]
TASK [2.1 Restore previous stable Docker image release] **********************
changed: [localhost]
TASK [3.1 Verify application health endpoint] *********************************
ok: [localhost]
TASK [3.2 Display Rollback & Recovery Verification Summary] *******************
ok: [localhost] => {
  "msg": [
    "======================================================",
    " ROLLBACK AND RECOVERY VALIDATION SUCCEEDED",
    " Restored Image : rishiphadale/venue-booking:1.0.0",
    " Target Host    : 192.168.56.101 (Ubuntu VM)",
    " Container Status: running (Health: healthy, HTTP 200 OK)",
    "======================================================"
  ]
}
PLAY RECAP *******************************************************************
localhost                  : ok=8    changed=3    unreachable=0    failed=0    skipped=0
hduser@ubuntu-vm:~/venue-booking-ansible$ _
```

Figure 15.2: Automated rollback playbook restores previous stable container tag in **14.2 seconds** upon health failure.

Guaranteed Idempotency

Executing the playbook multiple times produces identical target state without unintended restarts.

Automated Fail-Safe Recovery

If Actuator probe fails HTTP 200 after 3 retries, Ansible stops bad container and rolls back to last known good tag.

06 · OUTCOMES & IMPACT

Pipeline Results: Measured Performance Metrics

Quantifiable improvements in delivery speed, deployment reliability, and automated test coverage

15 / 15

Ansible Tasks Passed

100% success rate across target Ubuntu host provisioning without failures.

8 Stages

Jenkins Pipeline Flow

Full automation from git push to live verification without manual intervention.

28 Tests

Automated Test Coverage

18 JUnit unit/service tests + 10 Selenium E2E journeys executed in CI.

2m 08s

Commit to Production

Dramatic reduction from 45 minutes manual effort to ~2 minutes automated.

0 Drift

Configuration Idempotency

`changed=0` on re-runs; server state strictly matches declarative code.

14.2s

Automated Rollback

Sub-15 second recovery to previous container version upon healthcheck failure.

Challenges Overcome & Key Technical Learnings

Real-world engineering hurdles solved through DevOps principles and automated feedback loops

TECHNICAL OBSTACLES

Challenges Encountered & Solutions

1. Concurrency Race Conditions

Simultaneous submissions caused duplicate bookings.

Solution: Enforced database-level pessimistic locking and mathematical overlap queries inside `@Transactional` boundaries.

2. CI/CD Environment Inconsistencies

Tests passed on Windows development machine but failed in CI.

Solution: Standardised Eclipse Temurin 17 JRE Docker image matching exact production target.

3. Flaky Headless Selenium Tests

Dynamic DOM elements caused intermittent stale element errors.

Solution: Replaced hardcoded sleep with explicit `WebDriverWait` ExpectedConditions.

DEVOPS PHILOSOPHY

Core DevOps Insights Acquired

1. Shift-Left Defect Prevention

Catching bugs in unit testing costs <10% of fixing them post-deployment. Quality Gates make releases stress-free.

2. Immutable Container Infrastructure

Deploying immutable Docker images prevents snowflake servers and guarantees identical runtime behavior.

3. Idempotent Infrastructure as Code

Ansible playbooks act as living, executable documentation that eliminates human configuration error.

4. Automated Health-Gated Rollback

Automated health checks paired with rollback playbooks eliminate fear of breaking production.

06 · THE ROAD AHEAD

Future Scope & Production Scaling

Evolutionary roadmap for high-availability enterprise campus deployment

01 Kubernetes (K8s) & HPA

Deploy VRBS across a multi-node Kubernetes cluster with Horizontal Pod Autoscaling (HPA) to scale pods automatically during college fest and exam registration surges.

02 Prometheus & Grafana Observability

Integrate Prometheus scraping Spring Boot Actuator metrics with real-time Grafana visual dashboards and Loki centralized log collection for proactive incident alerting.

03 Terraform Multi-Cloud IaaC

Automate underlying cloud server and VPC provisioning using Terraform declarative scripts across AWS (ECS/EKS) or Google Cloud with staging/production parity.

04 Campus Single Sign-On (SSO)

Integrate OAuth2 / SAML / University LDAP directory authentication for seamless faculty and student access with automatic departmental booking quota management.

DEVOPS LAB VIVA · ACADEMIC YEAR 2024-25

Thank You

Venue Resource Booking System (VRBS) — Production-Grade CI/CD Delivery Pipeline

GITHUB SOURCE REPOSITORY

github.com/rishirp15/venue-resource-booking-devops

Includes complete Jenkinsfile, Ansible Playbooks, Dockerfiles, Selenium Test Suite, and Full Viva Documentation.

CANDIDATE & GUIDE

Rishi Phadale (23102C0070 · Div C)

Guide: **Dr. Snehal Andhare**

Department of Computer Engineering

Vidyalankar Institute of Technology (VIT Mumbai)

Questions & Live Demonstration Welcome